Steven R. Dunbar
Department of Mathematics
203 Avery Hall
University of Nebraska-Lincoln
Lincoln, NE 68588-0130
http://www.math.unl.edu
Voice: 402-472-3731
Fax: 402-472-8466

Topics in Probability Theory and Stochastic Processes
Steven R. Dunbar

The Game of Monopoly as a Markov Chain

Rating

Mathematically Mature: may contain mathematics beyond calculus with proofs.

Section Starter Question

While playing the game of Monopoly, how often do you "take a walk on the Boardwalk"? That is, what is the fraction of time spent on the most expensive property in the game? How would you estimate this fraction?

Key Concepts

1. Modeling the game of Monopoly as a Markov chain finds the stationary distribution to find the properties landed on most frequently.
2. The modeling proceeds as a sequence of increasingly detailed Markov chains, each modeling more of the rules of the game.
3. The transition probability matrix is composed as a product of simpler transition matrices, each modeling a specific type of moves around the board.

Vocabulary

1. A **circulant matrix** is a square matrix in which each row vector is rotated one element to the right relative to the preceding row vector.

Mathematical Ideas

Monopoly as a Markov Chain

This section assumes familiarity with the standard layout and rules of the board game Monopoly, in particular, the classic U.S. version based on Atlantic City NJ. The goal is to model the game as a Markov chain to find the stationary distribution to determine the properties a player token lands on most frequently. Those squares on the board are "more valuable" in the sense that other players will land on them more frequently and pay rent more frequently. Deciding which color group squares are "best" is more complicated because of the variable costs of rent and improvement costs around the board and that aspect is not considered here. The main intention of this section is to show how to successively make changes to a simple Markov chain to simulate a more nuanced model incorporating more transitions between states. Another intention is to analyze a Markov chain larger than introductory Markov chains.

The basic transition probability matrix around the board is simply determined by the rolls of two fair six-sided dice. However making distant jumps due to the "Go To Jail" square and the drawing of certain *Community Chest* and *Chance* cards complicate the transition probability matrix. To understand this complication, first consider a much smaller example with just 4 spaces: Jail is state 1, Policeman is state 2, Community Chest is state 3, and Go is state 4, see Figure 1. A fair two-sided "die" (i.e. a fair coin) determines the 1 or 2 step forward motion of a player token around the board. Then the "dice roll" transition matrix R is

$$R = \begin{pmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 \end{pmatrix}.$$

(The source of this section, [1], uses the alternative convention that P_{ij} represents the probability of moving *to* state i *from* state j. Thus all matrices here are the transpose of matrices in [1].) Note that this transition probability matrix is regular, and that the stationary distribution is $(1/4, 1/4, 1/4, 1/4)$.

The "Jail" matrix represents the transition from landing on the Police-

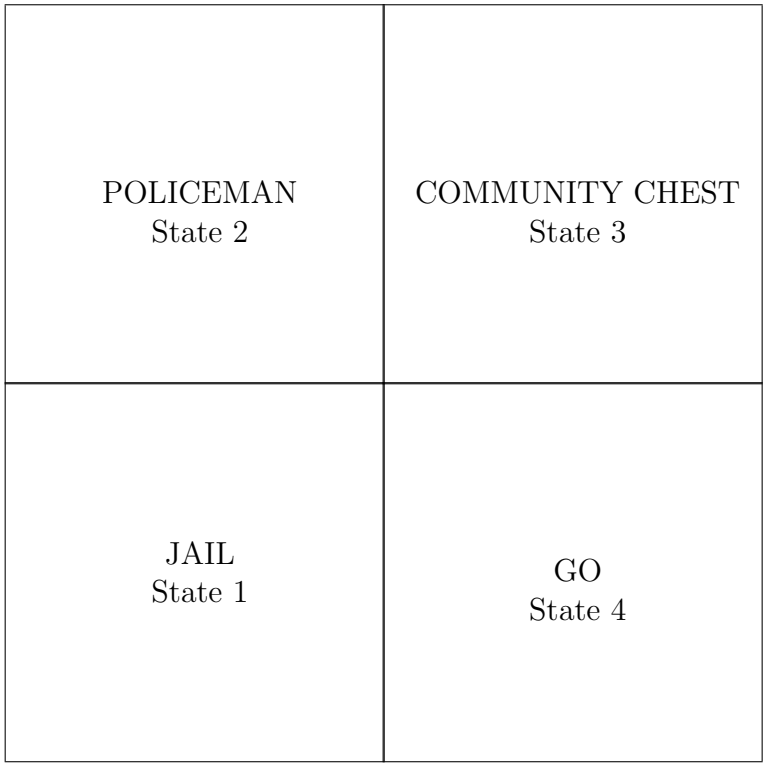

Figure 1: A Monopoly board with 4 spaces.

man:

$$J = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

This matrix differs from the identity matrix only in the second row (Policeman) where, with probability 1, a player goes directly to state 1, Jail.

The last element is the Community Chest space. Of the 16 Community Chest cards (including the Get Out of Jail Free card), 14 penalize or reward some peculiar accomplishment, e.g. \$10 for second prize in a beauty contest. However, one card sends the player to Jail, i.e. state 1, and the other card advances the player to Go, state 4. To form the Community Chest matrix, again start with the identity matrix and edit the third row to get

$$C = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{1}{16} & 0 & \frac{14}{16} & \frac{1}{16} \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The transition matrix for the simplified 4-space board is then computed by the product

$$RJC = \begin{pmatrix} \frac{17}{32} & 0 & \frac{7}{16} & \frac{1}{32} \\ \frac{1}{32} & 0 & \frac{7}{16} & \frac{17}{32} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

The order of the factors reflects the process of the play: A player rolls first and then follows the actions of where the token lands, i.e. go to Jail, or draw a Community Chest card. Now the second row of the matrix gives the transition probabilities for a token on the Policeman's square. However, this square is never occupied, the second column of the product is entirely 0. Note that the conditional probabilities of going from Policeman state 2 to other states are not 0. In fact, those conditional probabilities are exactly what would be expected starting from square 2. But the probability of being at the Policeman state is 0, so these correctly calculated probabilities are irrelevant. In terms of Markov chain states, the Go, Jail, and Community Chest states are communicating and the Policeman state is not accessible from the other states. Deleting the second column and row from the matrix

gives the transitions between the remaining 3 occupiable states. The stationary distribution is $(\frac{16}{27}, \frac{7}{27}, \frac{4}{27})$. The high frequency of landing in Jail in this truncated game is not surprising.

The states of the Markov chain representing the full game are the 40 spaces around the board, numbered 1 through 40 with Mediterranean Avenue as 1 and Go as 40.

The 40×40 Rolling matrix has as its first row

$$[0, 0, \frac{1}{36}, \frac{2}{36}, \frac{3}{36}, \frac{4}{36}, \frac{5}{36}, \frac{6}{36}, \frac{5}{36}, \frac{4}{36}, \frac{3}{36}, \frac{2}{36}, \frac{1}{36}, 0, 0, \ldots, 0].$$

The second row is a rotated version of this row, add a zero at the front and shift the nonzero entries one column to the right. Form each succeeding row the same way until eventually the first entry is nonzero, as the rolls start to wrap around the board. This creates a **circulant matrix**, a square matrix in which each row vector is rotated one element to the right relative to the preceding row vector. Considering just this transition probability matrix, the steady state distribution is uniformly $\frac{1}{40} = 0.025$ which is expected.

For now, continue to ignore the more complicated rules about exiting from the Jail square, and let the J matrix be just the identity matrix with the 1 in row 30 column, Policeman, moved from column 30 to column 10, Jail.

The Community Chest spaces or states are 2, 17, and 33. Community Chest cards are most likely to give you money leaving the state unchanged. But two of the cards move the player either to Jail, state 10 or to Go, state 40. To form the Community Chest matrix, Cc change the 1s on the diagonal in row 30 of the identity matrix to $\frac{14}{16}$ and enter $\frac{1}{16}$ in each of column 10, Jail, and column 40, Go.

The Chance spaces or states are 7, 22 and 36. A Chance card is more likely than a Community Chest card to move players, 9 of the 16 cards directing a player to a different state. The existence of cards such as "Go back three spaces" and "Advance to the nearest railroad" means that the transitions depend on which Chance a player lands on. The Chance matrix Ch differs from the identity in rows 7, 22 and 36. It makes sense to multiply the Community Chest matrix *after* the Chance matrix, since there is a probability $\frac{1}{16}$ of moving from Chance state 36 to Community Chest state 33 and thence to Go or Jail, each with probability $\frac{1}{16}$.

Finally, the full transition probability matrix is $P = RJChCc$. The steady state distribution gives a preliminary look at the proportion of time a

player spends in each state over the course of a long game. The steady state distribution as a bar graph is in Figure 2. Table 1 summarizes the results with the 14 most frequently visited states in this model. The R script to generate the first two columns of Table 1 from the steady state distribution, or principle eigenvector, of the transition probability matrix is in the Scripts section below. The last two columns come from a simulation with 10^6 steps. Again note that all states except 30 are communicating and the Policeman state 30 is not accessible from the other states. The frequencies computed here differs slightly from the corresponding table in [1]. The maximum difference between state frequencies is 0.001441622 at state 5 (Reading Railroad) with a relative error of 0.04937336 or about 5% compared to the frequency here. However, the maximum difference between the eigenvector frequencies and the simulation frquencies differ only by 0.0003050491 at state 19 (New York Avenue). Furthermore, the decreasing order of eigenvalue frequencies differs from the order of simulated frequencies in only 3 states, with a transpositions of states 6 and 7 and substitution of state 27 for state 23 at the lowest rank. This lends some confidence to the eigenvector and the simulation values. It is possible that [1] used an eigen package which was not as refined in 1996 as more recent eigen packages. It is also possible that they calculated the primary eigenvector with matrix powers, which could lead to numerical differences. It is also possible that they report the result of long-time simulations, also leading to numerical differences.

Modeling Rules for Jail

This first model and simulation ignores two finer aspects of the game. This model assumes that a player sent to Jail immediately pays the $50 fine or uses a Get Out of Jail card to exit Jail on the next roll. But late in a game, when the board is covered with hotels and houses with high rent, players may choose to stay in Jail for a couple of turns to avoid paying rents. The rules allow this for two turns unless doubles are rolled, in which case a player must leave, exposed to nearby high rents. Because one must leave Jail on the third roll, the model now needs a record of how many unsuccessful rolls of doubles have occurred. The solution is to define two new Jail states, one for being newly Jailed, now state 41 and the other for having failed at doubles exactly once, state 42. The original Jail state 10 serves as both the Just-Visiting and twice-failed-rolling-doubles states, since in both cases, the player leaves on the next turn. Table 2 gives the results of one simulation of this expanded

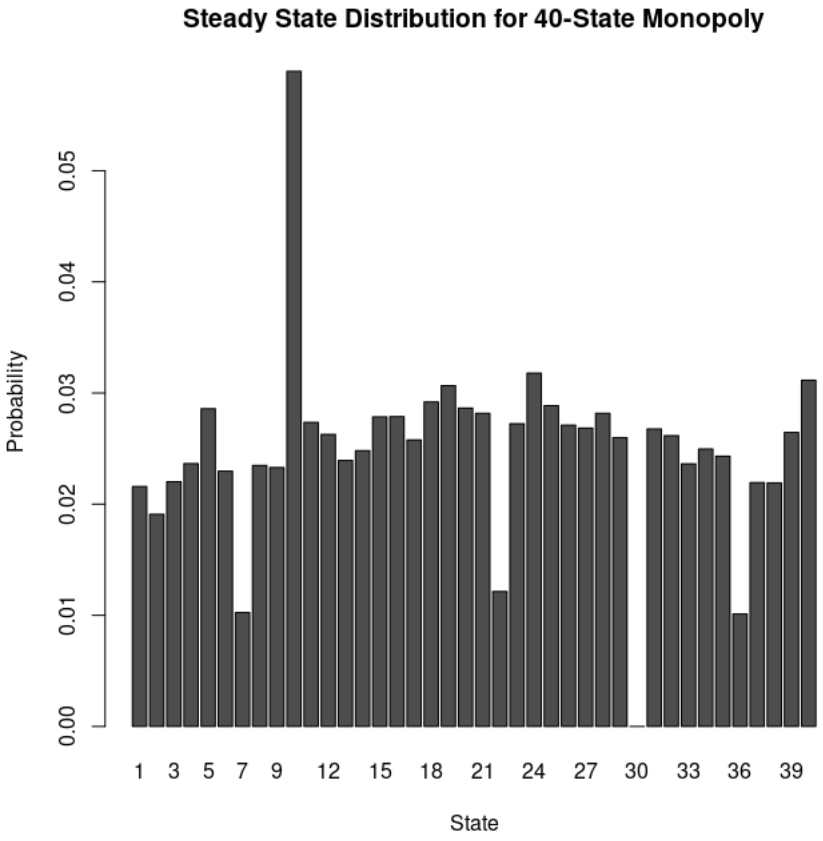

Figure 2: Steady state distribution for the first 40-state model of Monopoly.

	State	Frequency	Abbott State	Abbott Frequency	Simulation State	Simulation
1	10	0.05894178	10	0.05896	10	0.059076
2	24	0.03178711	24	0.03188	24	0.031750
3	40	0.03114815	40	0.03114	40	0.031057
4	19	0.03066205	19	0.03071	19	0.030357
5	18	0.02919838	25	0.03064	18	0.029362
6	25	0.02886396	5	0.02993	20	0.028946
7	20	0.02864452	15	0.02919	25	0.028922
8	5	0.02858912	18	0.02917	5	0.028883
9	28	0.02817410	20	0.02875	28	0.028228
10	21	0.02816908	21	0.02830	21	0.028136
11	16	0.02787877	28	0.02811	16	0.027910
12	15	0.02785248	16	0.02777	15	0.027813
13	11	0.02735978	23	0.02739	11	0.027455
14	23	0.02723961	11	0.02736	27	0.027314

Table 1: The 14 most-frequently visited states in the simple 40-state model.

Markov chain with the Jail states 10, 41 and 42 combined. State 10 now is a state reached by either a roll of the dice or by aging out of Jail so a player can move on to other states. The steady state distribution of this expanded Markov chain as a bar graph is in Figure 3, also with the Jail states 10, 41, and 42 combined.

The specific order of most frequently visited states is not as important as it appears in Table 2. Differences in frequency generally appear in the third significant digit, so approximation errors may account for differences. More useful is to consider *sets of most frequently visited states*. The most frequently visited set of states is 10, 41, 42, 24 and 40 in both calculations. But consider states 19, 18, 20, 25, and 5 appearing respectively as the 4th through 8th most visisted states using the scripts here, but states 25, 5, 20, 19, 18 appearing respectively as the 4th through 8th most visisted states in Abbott and Richey's calculations. The specific order is almost reversed but the sets are the same.

There is still one more detailed rule to consider, any player who rolls three doubles in a row goes to Jail. One way to handle this detail is to add 80 new states to the model as before, keeping track of history in a model with a total

	State	Frequency	Abbott State	Abbott Frequency	Simulation State	Simulation
1	10, 41, 42	0.107935954	10, 41, 42	0.10800	10, 41, 42	0.10690
2	24	0.030042598	24	0.03013	24	0.02999
3	40	0.029496065	40	0.02949	40	0.02971
4	19	0.028168790	25	0.02905	18	0.02811
5	18	0.028150572	5	0.02845	19	0.02805
6	20	0.028117861	20	0.02822	25	0.02793
7	25	0.027292567	19	0.02821	20	0.02739
8	5	0.027180292	18	0.02812	5	0.02709
9	16	0.026846759	16	0.02674	16	0.02708
10	28	0.026765686	28	0.02669	12	0.02679
11	12	0.026365350	15	0.02659	21	0.02672
12	21	0.026150719	12	0.02636	28	0.02669
13	11	0.026026695	21	0.02628	39	0.02636
14	23	0.025703433	11	0.02603	26	0.02600

Table 2: The 14 most-frequently visited states in the 42 state model with Jail states 10, 41 and 42 combined.

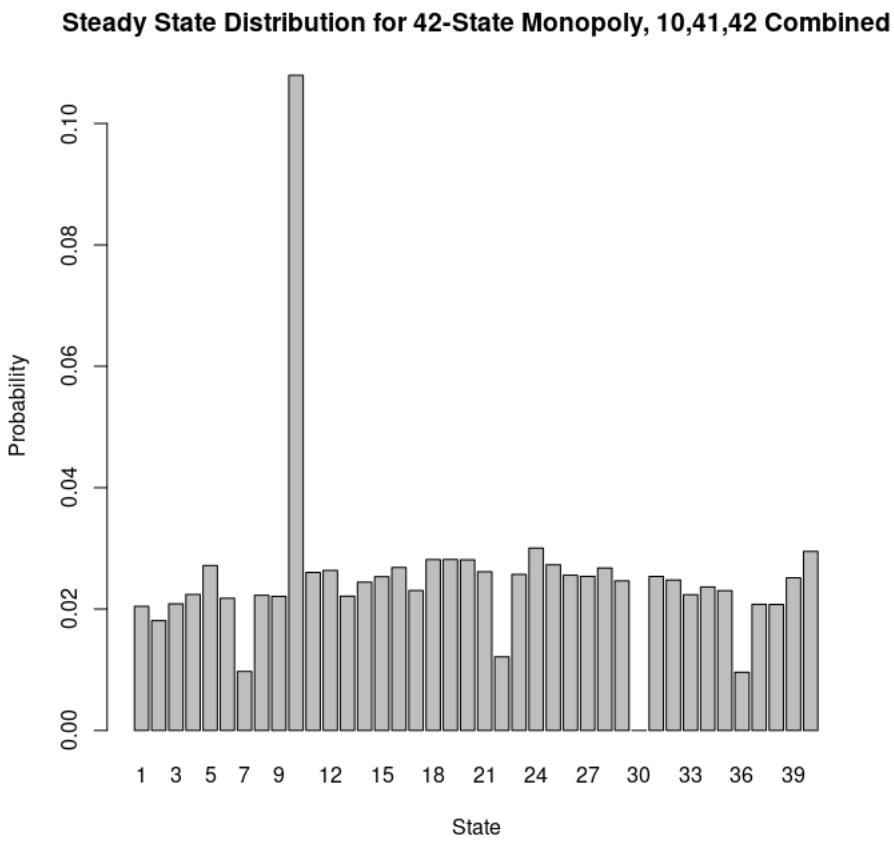

Figure 3: Steady state distribution for the expanded 42-state model of Monopoly with Jail states 10, 41 and 42 combined.

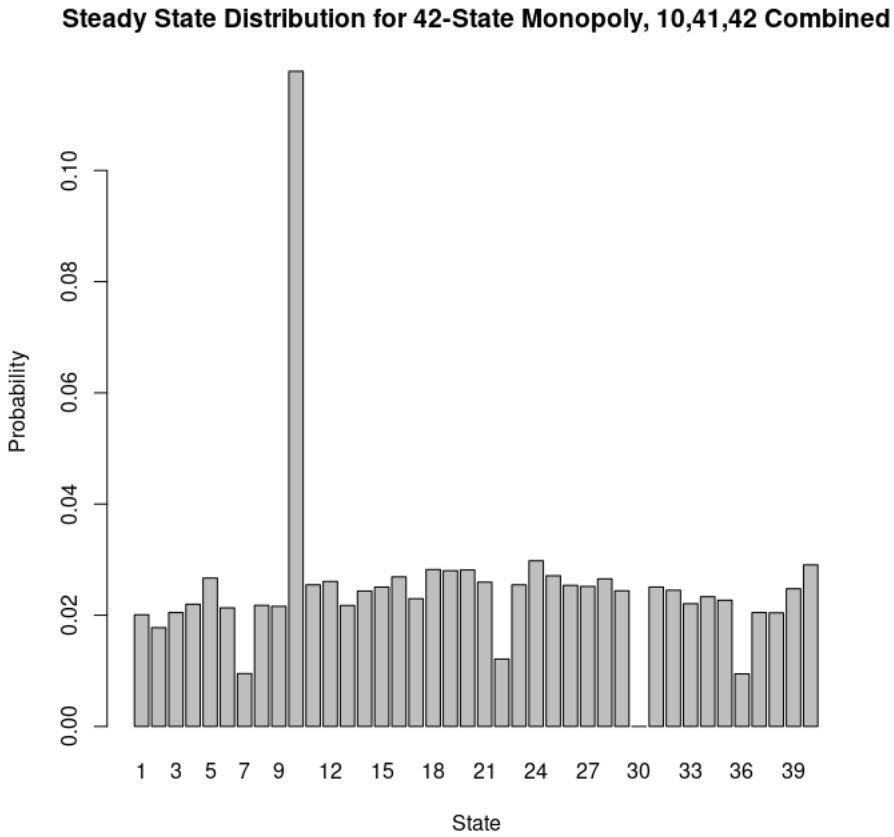

Figure 4: Steady state distribution for the final 42-state model of Monopoly.

of 122 states. An alternative way to handle this is to make an approximation by adjusting the Jail matrix J so that it sends a player not already in Jail to Jail with probability 1/216 since the probability of rolling three doubles in a row is $\frac{1}{216}$. Replace each 1 on the diagonal with $\frac{215}{216}$ and add $\frac{1}{216}$ to column 41. After making these last adjustments to the probability transition matrix, Table 3 summarizes the results of simulation. The steady state distribution for this final Markov chain model as a bar graph is in Figure 4.

	State	Frequency
1	10	0.117841272
2	24	0.029795914
3	40	0.029055135
4	18	0.028235759
5	20	0.028148582
6	19	0.028019441
7	25	0.027108791
8	16	0.026894000
9	5	0.026694388
10	28	0.026549407
11	12	0.026091348
12	21	0.025963983
13	11	0.025481659
14	23	0.025471064
15	26	0.025384224
16	27	0.025192088
17	31	0.025076589
18	15	0.025064206
19	39	0.024766786
20	32	0.024499247
21	29	0.024398944
22	14	0.024354843
23	34	0.023334506
24	17	0.022984619
25	35	0.022713579
26	33	0.022086039
27	4	0.021986015
28	8	0.021786854
29	13	0.021756287
30	9	0.021606785
31	6	0.021335917
32	37	0.020478819
33	3	0.020476878
34	38	0.020438135
35	1	0.020087639
36	2	0.017758354
37	22	0.012115464
38	7	0.009514394
39	36	0.009452046
40	30	0.000000000

Table 3: The frequency of visited states in the final 42-state model with the approximation for going to Jail by rolling three doubles.

Additional Modeling Considerations

A few modeling assumptions are not explicitly considered in this model. One assumption is that Community Chest and Chance card decks are shuffled after each draw so that drawing the cards is a random event, possibly even repeating. The game actually has 17 Community Chest cards including a "Get Out of Jail Free" card, which can be saved by the player who draws it. Assuming that card has been drawn and retained gives the denominator of 16 used in the probabilities of the Community Chest matrix. Likewise, Monopoly uses 16 Chance cards but one is another "Get Out of Jail Free" card which can be saved. Assuming that card is still in the Chance deck gives the denominator of 16 used in the probabilities of the Chance matrix. This pair of mixed assumptions makes uniform denominators in the two matrices. Also, the final model assumes that appearance of a third consecutive roll of doubles is equally likely to occur at any space on the board, which is not strictly correct. Some squares are already known to be visited more than others and a third consecutive doubles could occur at an even number of squares further on the board from where the doubles run began.

In [1], Abbott and Richey use the steady state distribution to find the expected rental value and break-even times of the various color group properties. However, since illustration of the modeling and simulation of large Markov chains is the focus of these notes, these notes do not follow that investigation.

Many sites on the Internet make essentially the same Markov chain analysis of Monopoly in varying degrees of detail and in slightly different ways. For example, some use a 120-state chain to model the "rolling doubles" rule, and then condense the chain to the 40 board states. All reach essentially the same conclusions. Many of those sites go further and make an expected Return on Investment analysis to determine the most valuable spaces on the board, not just the most frequently visited. The Return on Investment analysis is not pursued here because the intention is to illustrate the Markov chain aspects.

Section Ending Answer

Since the board has 40 spaces, a first guess is that a token spends about $1/40 = 0.025$ of the time on Boardwalk. In fact, using the most detailed Markov chain here, 0.02431 of the time is spent at Boardwalk. It is less than the easy estimate because the model shows that a much larger fraction of time is spent in Jail, and in fact, not all spaces are equally likely to be occupied.

Sources

The subsection on Monopoly as a Markov chain is based on [1] with more information from [2]. The simulation scripts is adapted from *Efficient R*, Chapter 7 by Colin Gillespie and Robin Lovelace.

Algorithms, Scripts, Simulations

Algorithm

```
Monopoly Game Markov Chain
    Comment Post: Steady state distribution
    of Markov chains modeling the Monopoly game
    Load Markov chain library
    ROTVEC
    Cyclically rotate vector elements one place to right

    Set number of states and state names

    Fill diceRoll matrix

    Fill diagonal Jail matrix with move from Police to Jail

    Fill diagonal Community Chest matrix with moves to other states

    Fill diagonal Chance matrix with moves to other states

    Transition probability matrix as product of
    diceRoll, Jail, Chance and Community Chest
    Create Monopoly Markov chain object
    Find steady state distribution of Markov Chains
    Report selected results in table or graph
```

Simulate Monopoly Game

```
Comment Post: Empirical stationary distributions
from simulation of the Monopoly game
moveSquare
Simulate dice roll and
keep track of jail time
return new state and jail time

checkState
Cyclically reduce state about state 40,
return state number

updateStateVector
Keep track of number of times a state is visited
return updated landings

communityChest
Move state according to random selection
of Community Chest card
return new state and jail time

Chance
Move state according to random selection
of Chance card,
return new state and jail time

mySimulateMonopoly
Initialize landings vector to hold occupancy of states
Initialize jailTime
Initialize first state at Go
for i = 1 to numberOfTurns
Move by moveSquare
if on Chance square, move by chance
if on Community Chest square, move by communityChest
Update landings
```

Scripts

R R script for 40-state Monopoly.

```r
library("markovchain")

rotvec <- function(vec) vec[ c(length(vec), 1:(length(vec)-1)) ]

nStates <- 40
monopolyStates <- as.character( c(1:nStates) )

diceRoll <- matrix(0, nStates, nStates)
p1 <- c(0, 0,
        1/36, 2/36, 3/36, 4/36, 5/36, 6/36, 5/36, 4/36, 3/36, 2/36, 1/36,
        numeric(nStates-13)
        )
for (i in 1:nStates) {
    diceRoll[i, ] <- p1
    p1 <- rotvec(p1)
}

Jail <- diag(nStates)
Jail[30,30] <- 0
Jail[30, 10] <- 1

Ch <- diag(nStates)
chanceStates <- c(7, 22, 36)
chanceDests <- c(5, 10, 11, 24, 39, 40)
Ch[ cbind( chanceStates, chanceStates ) ] <- 7/16
Ch[ cbind( chanceStates, chanceStates-3 ) ] <- 1/16 #Go Back 3 spaces
Ch[ chanceStates, chanceDests] <- 1/16
Ch[7,12] <- 1/16; Ch[22,28] <- 1/16; Ch[36, 12] <- 1/16 #nearest utility
Ch[7,15] <- 1/16; Ch[22,25] <- 1/16; #nearest Railroad,
Ch[36, 5] <- 2/16 # note 2 ways to get to Reading Railroad

Cc <- diag(nStates)
chestStates <- c(2, 17, 33)
chestDests <-  c(10, 40)
Cc[ cbind(chestStates, chestStates) ] <- 14/16
Cc[chestStates, chestDests] <- 1/16

```

```r
monopolyMatrix <- diceRoll %*% Jail %*% Ch %*% Cc

mcMonopoly <- new("markovchain", states = monopolyStates, byrow = TRUE,
                  transitionMatrix = monopolyMatrix, name = "Monopoly")
ss <- steadyStates(mcMonopoly)

top14ofAbbott <- (1/100)*c(5.896, 3.188, 3.114, 3.071, 3.064, 2.993, 2.919, 2.917, 2.875, 2.830, 2.811, 2.777, 2.739, 2.736)
top14ofAbbottIndices <- c(10, 24, 40, 19, 25, 5, 15, 18, 20, 21, 28, 16, 23, 11)

sstop14 <- sort(ss, decreasing=TRUE)[1:14]
sstop14Indices <- order(ss, decreasing=TRUE)[1:14]

maxDiff <- max( abs( sstop14 - top14ofAbbott) )
cat("Maximum Difference: ", maxDiff, "\n")
maxDiffIndex <- which.max( abs( sstop14 - top14ofAbbott) )
cat("Maximum Difference Index: ", maxDiffIndex, "\n")
maxRelDiff <- maxDiff/sstop14[maxDiffIndex]
cat("Relative Maximum Difference: ", maxRelDiff, "\n")

ssdf <- data.frame( sstop14Indices, sstop14)

barplot(ss, names.arg=1:40,
        main="Steady State Distribution for 40-State Monopoly",
        xlab="State", ylab="Probability")
```

R script for 42-state Monopoly.

```r
  library("markovchain")

rotvec <- function(vec) vec[ c(length(vec), 1:(length(vec)-1)) ]

nStates <- 42
monopolyStates <- as.character( c(1:nStates) )

diceRoll <- matrix(0, (nStates-2), (nStates-2))
p1 <- c(0, 0,
```

```
        1/36, 2/36, 3/36, 4/36, 5/36, 6/36, 5/36, 4/36, 3/36, 2/36, 1/36,
        numeric((nStates-2)-13)
        )
for (i in 1:(nStates-2)) {                #regular states 1 through 40
    diceRoll[i, ] <- p1
    p1 <- rotvec(p1)
}
## Next, no probability of going from board state to extra Jail states
diceRoll  <- cbind(diceRoll, matrix(0, (nStates-2), 2))
## add rows from extra Jail states
diceRoll  <- rbind(diceRoll, matrix(0, 2, nStates))
## fill transitions for rolling doubles to get out of Jail
diceRoll[41, 41] <- 0; diceRoll[41, 42] <- 5/6 #stay in jail one turn
diceRoll[41, c(12, 14, 16, 18, 20, 22)] <- 1/36 #roll doubles
diceRoll[42, 42] <- 0; diceRoll[42, 10] <- 5/6 #stay in Jail one more turn
diceRoll[42, c(12, 14, 16, 18, 20, 22)] <- 1/36 #roll doubles

Jail <- diag(nStates)
Jail[30,30] <- 0
Jail[30, 41] <- 1                       #go from Police to Jail

Cc <- diag(nStates)
chestStates <- c(2, 17, 33)
chestDests <-  c(40, 41)
Cc[ cbind(chestStates, chestStates) ] <- 14/16
Cc[chestStates, chestDests] <- 1/16

Ch <- diag(nStates)
chanceStates <- c(7, 22, 36)
chanceDests <- c(5, 11, 24, 39, 40, 41)
Ch[ cbind( chanceStates, chanceStates ) ] <- 7/16
Ch[ cbind( chanceStates, chanceStates-3 ) ] <- 1/16 #Go Back 3 spaces
Ch[ chanceStates, chanceDests] <- 1/16
Ch[7,12] <- 1/16; Ch[22,28] <- 1/16; Ch[36, 12] <- 1/16 #nearest utility
```

```
Ch[7,15] <- 1/16; Ch[22,25] <- 1/16; #nearest Railroad,
Ch[36, 5] <- 2/16 # note 2 ways to get to Reading Railroad

monopMat <- diceRoll %*% Jail %*% Ch %*% Cc

mcMonopoly42 <- new("markovchain", states = monopolyStates, byrow = TRUE,
                 transitionMatrix = monopMat, name = "Monopoly")
ss <- steadyStates(mcMonopoly42)

ssComb  <- ss
ssComb[10]  <- ssComb[10] + ssComb[41] + ssComb[42]
ssCombTop14 <- sort(ssComb[1:40], decreasing=TRUE)[1:14]
ssCombTop14Indices <- order(ssComb[1:40], decreasing=TRUE)[1:14]
ssCombdf <- data.frame( ssCombTop14Indices, ssCombTop14)

top14ofAbbott <- (1/100)*c(10.800, 3.013, 2.949, 2.905, 2.845, 2.822, 2.821, 2.674, 2.669, 2.659, 2.659, 2.636, 2.628, 2.603)
top14ofAbbottIndices <- c(10, 24, 40, 25, 5, 20, 19, 18, 16, 28, 15, 12, 21 , 11)

maxDiff <- max( abs( ssCombTop14 - top14ofAbbott) )
cat("Maximum Difference: ", maxDiff, "\n")
maxDiffIndex <- which.max( abs( ssCombTop14 - top14ofAbbott) )
cat("Maximum Difference Index: ", maxDiffIndex, "\n")
maxRelDiff <- maxDiff/ssCombTop14[maxDiffIndex]
cat("Relative Maximum Difference: ", maxRelDiff, "\n")

barplot(ssComb[1:40], names.arg=1:40,
        main="Steady State Distribution for 42-State Monopoly, 10,41,42 Combined",
        xlab="State", ylab="Probability")
```

R script to simulate 42-state Monopoly.

```
  ##  This function calculates where you will land on your next move.
moveSquare <- function(current, jailTime) {
    dice <- sample(seq(1,6), 2, replace=TRUE)
```

```

    if (current == 10 & jailTime > 0) { # in Jail
        if (dice[1] == dice[2]) {           # roll doubles!
            current <- 10 + sum(dice)       # get out of Jail
            jailTime <- 0
        } else {                            #don't move but...
            jailTime <- (jailTime + 1) %% 3 # count up jail time
        }
    } else {
        current <- current + sum(dice)      # regular roll
    }
  return(c(current, jailTime))
}

## Helper function to avoid code replication
checkState <- function(current) {
  if (current > 40) {
    current <- current - 40
  } else if (current < 1) {
    current <- current + 40
  }
  return(current)
}

## Helper function to record landings,
updateStateVector <- function(current, jailTime, landings) {
    if (current == 10 & jailTime > 0) {
        landings[40 + jailTime] = landings[40 + jailTime] + 1
    } else {
        landings[current] <- landings[current] + 1
    }

    return(landings)
}

## Move according to community chest cards
## Use 16 cards, Get Out of Jail is reserved
communityChest <- function(current, jailTime) {
```

```
  u <- runif(1)
  goto <- current # Default. Do nothing
  if (u < 1 / 16) {
    goto <- 40 # Go
  } else if (u < 2 / 16) {
      goto <- 10 # Jail
      jailTime <- 1
  }
  return( c(goto, jailTime) )
}

## Move according to chance cards
## Use 16 cards, Get Out of Jail is NOT reserved
chance <- function(current, jailTime) {
  u <- runif(1)
  goto <- current # Default. Do nothing
  if (u < 1 / 16) {
      goto <- 40 # Go
  } else if (u < 2 / 16) {
      goto <- 24 # Illinois Ave.
  } else if (u < 3 / 16) {
      goto <- 11 # St. Charles Place
  } else if (u < 4 / 16) {
      goto <- 10 # Jail
      jailTime <- 1
  } else if (u < 5 /16 ) {
      goto <- 5 #Take ride on Reading
  } else if (u < 6 / 16) {
      goto <- 39 # Boardwalk
  } else if (u < 7 / 16) {
      goto <- checkState(current - 3) # Must check, since goto maybe negative!
  } else if (u < 8 / 16) {
      if (current > 28  | current < 12) { # Advance to nearest Utility
      goto <- 12 # Water Works
    } else {
      goto <- 28 # Electricity
    }
  } else if (u < 9 / 16) { # Advance to nearest Railroad
    if (current > 35  | current < 5) {
      goto <- 5                         # Reading
    } else if (current > 5  & current < 15) {
      goto <- 15 #Pennsylvania
    } else if (current > 15  & current < 25) {
```

```
      goto <- 25 # B & O
    } else if (current > 25  & current < 35) {
      goto <- 35 # Short Line
    }
}
  return(c(goto, jailTime))
}

## Main, record where landings occur
mySimulateMonopoly <- function (numberOfTurns)
{
    landings <- numeric(42) # vector to hold occupancy of states,
    # 1 is Med Ave, 40 is Go
    # 41 is in Jail for one turn, 42 is Jail for two turns
    # 10 is Just Visiting Jail or Jail for three turns, ready to get out
    jailTime <- 0
    current <- 40           # start at Go
    for (i in 1:numberOfTurns) {
        moveState <- moveSquare(current, jailTime)
        current <- moveState[1]
        jailTime <- moveState[2]
        current <- checkState(current)
        if (current == 30) {
            current <- 10
            jailTime <- 1
        }
        if (current == 7 | current == 22 | current == 36) {
            move <- chance(current, jailTime)
            current <- move[1]
            jailTime <- move[2]
        }
        if (current == 2 | current == 17 | current == 33) {
            move <- communityChest(current, jailTime)
            current <- move[1]
            jailTime <- move[2]
        }
        landings <- updateStateVector(current, jailTime, landings) # must do update
    }
return(landings)
```

```
}

n  <- 1000
ss  <- mySimulateMonopoly(n)/n
```

Problems to Work for Understanding

1: Create a small Monopoly game with just 8 spaces:

1. Reading Railroad is state 1,
2. Jail is state 2,
3. Pennsylvania Railroad is state 3,
4. Policeman is state 4,
5. B. & O. Railroad is state 5,
6. Chance is state 6,
7. Short Line is state 7 and
8. Go is state 8.

A fair two-sided "die" (i.e. a fair coin) determines the 1 or 2 step forward motion of a player token around the board. Considering this as a Markov Chain, find the stationary distribution. Assume the simple rule that a player leaves Jail immediately, no staying for 2 more turns. Be sure to include the effect drawing an "Advance to the Nearest Railroad", "Take a Ride on the Reading", "Go to Jail", "Advance to Go", or "Go back 3 spaces" card from the Chance deck. All other Chance movement cards (e.g. Illinois Avenue, Boardwalk, Utility) have no effect on movement.

Reading Suggestion:

References

[1] Steve Abbott and Matt Richey. Take a Walk on the Boardwalk. *The College Mathematical Journal*, 28(3):162–171, May 1997.

[2] Robert Ash and Richard Bishop. Monopoly as a Markov Process. *Mathematics Magazine*, 45:26–29, 1972.

Outside Readings and Links:

1.

2.

3.

4.

Solutions

1: Use the following script:

```
    library("markovchain")

rotvec <- function(vec) vec[ c(length(vec), 1:(length(vec)-1)
    ) ]

nStates <- 8
monopolyStates <- as.character( c(1:nStates) )

diceRoll <- matrix(0, nStates, nStates)
p1 <- c(0, 1/2, 1/2, 0, 0, 0, 0, 0)

for (i in 1:nStates) {
    diceRoll[i, ] <- p1
    p1 <- rotvec(p1)
}

Jail <- diag(nStates)
Jail[4, 4] <- 0
Jail[4, 2] <- 1                        #go from Police to Jail

Ch <- diag(nStates)
Ch[6, 7] <- 1/16                          #Nearest Railroad
Ch[6, 8] <- 1/16                          #Advance to Go
Ch[6, 1] <- 1/16                          #Reading Railroad
Ch[6, 2] <- 1/16                          #Go to Jail
Ch[6, 3] <- 1/16                          #Go back 3
Ch[6, 6] <- 11/16                          #All other cards

monopMat <- diceRoll %*% Jail %*% Ch

mcMonopoly8 <- new("markovchain", states = monopolyStates,
    byrow = TRUE,
                    transitionMatrix = monopMat, name = "
    Monopoly")
ss <- steadyStates(mcMonopoly8)
cat(ss, "\n")
```

The steady state distribution is	1	0.07731016
	2	0.383806
	3	0.2342177
	4	0.0
	5	0.1171089
	6	0.04025618
	7	0.08234218
	8	0.06495883

I check all the information on each page for correctness and typographical errors. Nevertheless, some errors may occur and I would be grateful if you would alert me to such errors. I make every reasonable effort to present current and accurate information for public use, however I do not guarantee the accuracy or timeliness of information on this website. Your use of the information from this website is strictly voluntary and at your risk.

I have checked the links to external sites for usefulness. Links to external websites are provided as a convenience. I do not endorse, control, monitor, or guarantee the information contained in any external website. I don't guarantee that the links are active at all times. Use the links here with the same caution as you would all information on the Internet. This website reflects the thoughts, interests and opinions of its author. They do not explicitly represent official positions or policies of my employer.

Information on this website is subject to change without notice.

Steve Dunbar's Home Page, `http://www.math.unl.edu/~sdunbar1`

Email to Steve Dunbar, `sdunbar1 at unl dot edu`

Last modified: Processed from LaTeX source on August 20, 2020